Data Classification Framework & Strategy — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Data Classification Framework & Strategy (Microsoft Purview) 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

What is delivered . . . 5

Business outcome . . . 5

Important boundaries . . . 6

1. Solution Overview — Data Classification Framework & Strategy (Microsoft Purview) 6

What this solution is . . . 6

Why this matters . . . 6

The classification tier model (starting point — confirm with your governance owners) . . . 7

The crosswalk (the core deliverable — one row per tier) . . . 7

Naming convention (used in every doc and script) . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Data Classification Framework & Strategy 9

2.1 Licensing for labels, DLP, and retention . . . 9

2.2 Least-privilege roles across all three control families . . . 9

2.3 Security & Compliance PowerShell connectivity . . . 10

2.4 An agreed tier model and classification standard (governance input) . . . 10

2.5 In-scope workloads and the classifying conditions per tier . . . 10

2.6 A pilot ring and a test/simulation-first commitment . . . 11

2.7 Rights-management (encryption) prerequisites for the protected tiers . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Data Classification Framework & Strategy 13

Step 1 — Assess the current classification posture (Assess) . . . 14

Step 2 — Design the tier model and the crosswalk (Design) . . . 14

Step 3 — Build the tier sensitivity labels (Configure, -WhatIf first) . . . 14

Step 4 — Publish the tier labels to the pilot ring (Pilot) . . . 15
Step 5 — Build the tier-aligned DLP baseline **in test mode** (Configure / test) . . . 15
Step 6 — Build the tier-aligned retention labels (Configure, -WhatIf first) . . . 15
Step 7 — Publish retention labels to the pilot ring (Pilot) . . . 16
Step 8 — Validate the crosswalk is coherent (Validate) . . . 16
Step 9 — Measure classification coverage (Validate) . . . 16
Step 10 — Expand and enforce (Expand) . . . 16
5. Testing & Validation Guide — Data Classification Framework & Strategy 17
Test 1 — Tier labels created as designed . . . 17
Test 2 — Tier labels visible to pilot users only . . . 17
Test 3 — Protected tiers actually encrypt . . . 17
Test 4 — DLP baseline stays in test mode until accepted . . . 18
Test 5 — DLP baseline matches the right content . . . 18
Test 6 — Tier retention labels publish and apply (pilot) . . . 18
Test 7 — Retention *behavior* is safe (controlled disposition test) . . . 18
Test 8 — The crosswalk is coherent (no orphans) . . . 19
Test 9 — The live crosswalk matches the standard . . . 19
Test 10 — Coverage evidence before enforcement . . . 19
Test 11 — Full readiness before expansion . . . 19
6. Troubleshooting Guide — Data Classification Framework & Strategy 19
Connectivity & permissions . . . 20
Labels not visible or not protecting . . . 20
DLP baseline issues . . . 21
Retention issues . . . 21
Crosswalk / coverage . . . 22
7. Best Practices — Data Classification Framework & Strategy 22
Tier model design . . . 22
The crosswalk (the core discipline) . . . 23
Protection & DLP . . . 23
Retention (defer the determination) . . . 23
Program & change management . . . 24
Security & safety hygiene . . . 24
8. Deliverables Summary — Data Classification Framework & Strategy 24
What you receive . . . 24
Documentation . . . 24
PowerShell tools (5) . . . 25

Outcomes 26
Explicitly out of scope (deferred) 26
Appendix A — PowerShell Scripts 28

Executive Summary — Data Classification Framework & Strategy (Microsoft Purview)

The problem

Most organizations classify data inconsistently — a sensitivity label means one thing, a DLP rule another, a retention policy a third — so protection has gaps, users get contradictory signals, and no one can prove what is covered. Inconsistent classification quietly undermines every downstream control: DLP, encryption, and data lifecycle all key off a classification scheme that, in practice, doesn't exist as a single agreed thing. It is both a compliance gap and a data-risk exposure.

The solution

A complete, self-guided **data classification framework** for Microsoft Purview — one scheme across the whole organization. It delivers a small, defensible set of **sensitivity tiers**, a written **classification standard** (tier definitions + handling rules), and the core artifact: a **crosswalk** mapping each tier to exactly one **sensitivity label**, one **DLP posture**, and one **retention treatment**, so all three control families finally agree. The framework is configured as a coherent **reference implementation** — the tier labels, a tier-aligned DLP baseline in **test** mode, and tier-aligned retention labels — every high-impact change **piloted in test/simulation before enforcement**. It uses only the Microsoft Purview portal and Security & Compliance PowerShell — no third-party tooling.

How it works

The engagement follows one disciplined model: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce).** Assess the current posture; design the tier model and crosswalk (the standard); build and publish the tier labels to a **pilot ring**; build the tier-aligned DLP baseline **in test mode**; build and publish tier retention labels; validate that every tier maps coherently (no orphans) and that the live crosswalk matches the standard; measure coverage; then expand tenant-wide and enforce DLP only after its test-mode reports are accepted. Five PowerShell tools — read-only or `-WhatIf` by default — automate discovery, configuration, the live crosswalk export, coherence validation, and coverage.

What is delivered

- Six implementation documents (Overview, Requirements, Implementation, Testing, Best Practices, Troubleshooting) plus this summary and a classification-standard crosswalk.
- Five Security & Compliance PowerShell tools: posture assessment, framework config/remediation helper (labels + DLP-in-test + retention), live crosswalk export, coherence validation, and coverage.
- A named, gated, reversible rollout with explicit pass/fail criteria at every step.

Business outcome

One classification scheme across the org — a defensible tier model, a written standard, and a crosswalk that makes coverage provable because every tier has exactly one label, one DLP posture, and one retention treatment. Classification that is consistent by design, piloted before it disrupts anyone, and

evidenced — at a fraction of the cost of a bespoke consulting engagement.

Important boundaries

The **tier model, DLP enforcement decisions, retention schedules, records declarations, and regulatory applicability** are **determinations owned by your governance, compliance, records, privacy, and legal functions** — this solution provides the framework method, the crosswalk, and the technical configuration + evidence, not the determination. Several capabilities (advanced DLP conditions, activity/content explorer coverage reporting, auto-labeling) are **licensing-dependent** (E5 / E5 Compliance) and the Purview portal is evolving — availability and paths are flagged "verify your licensing/tenant," never assumed. The deep auto-labeling, DLP, and retention-schedule builds are separate engagements that plug into this framework.

1. Solution Overview — Data Classification Framework & Strategy (Microsoft Purview)

What this solution is

A **data classification framework** is the single, organization-wide scheme that says *how many sensitivity tiers exist, what each one means, and which technical controls enforce it*. This solution delivers that framework end to end in Microsoft Purview: a small, defensible set of **sensitivity tiers**, a written **classification standard** (tier definitions + handling rules), and — the core artifact — a **crosswalk** that maps each tier to its **sensitivity label**, its **DLP posture**, and its **retention treatment**, so all three control families key off *one* scheme instead of drifting apart. The framework is configured as a **reference implementation** (the tier labels, a tier-aligned DLP baseline in **test/simulation** mode, and tier-aligned retention labels), piloted, validated for coherence, then expanded — using only the Microsoft Purview portal and Security & Compliance PowerShell (no third-party tooling).

This is the **framework and strategy** layer — the tier model, the standard, and the tier→controls crosswalk that everything else aligns to. It is deliberately distinct from the deep *build* solutions: the exhaustive label taxonomy/auto-labeling build, the full DLP policy development, and the records retention schedule are each their own engagement. Here you establish the **scheme** and a coherent minimum reference implementation of it; those solutions go deep on each control family.

Why this matters

- **Inconsistent classification undermines every control.** If a "Confidential" label, a DLP rule, and a retention policy each define "sensitive" differently, protection has gaps and users get contradictory signals. One framework removes the ambiguity.
- **The crosswalk is the contract.** When every tier has exactly one label, one DLP posture, and one retention treatment, you can *prove* coverage — and spot the tier that has a label but no DLP, or retention but no label.
- **A framework scales; ad-hoc labels don't.** New DLP policies, new workloads, and Copilot/data-governance initiatives all plug into the tier model instead of inventing their own definitions.

• **High-impact controls are piloted first.** DLP blocking, encryption, and retention deletion are disruptive if rolled out carelessly — the framework's reference implementation is created in **test/simulation** and piloted before any enforcement.

The classification tier model (starting point — confirm with your governance owners)

*The four-tier model below is a **technical starting point**. The final number of tiers, their names, definitions, and handling rules are a **governance / compliance / records determination** (see §2 and the deferral note). The names here are reused verbatim across every doc and script.*

Tier	Name	Example definition (confirm with governance)	Example handling rule
Tier 0	`Public`	Approved for public release; no harm if disclosed.	May be shared externally.
Tier 1	`Internal`	Default for all business content; not for public release.	Share inside the org; external sharing by exception.
Tier 2	`Confidential`	Sensitive business data, PII, or contract-restricted data.	Encrypt; restrict external sharing.
Tier 3	`Restricted`	Highly regulated / high-impact if disclosed (e.g. regulated PII, secrets).	Encrypt + mark; named-recipient only; strong DLP.

The crosswalk (the core deliverable — one row per tier)

Each tier maps to exactly one artifact in each control family. Names are reused verbatim everywhere.

Tier	Sensitivity label	DLP posture (piloted in **test** first)	Retention treatment (records/legal determination)
Tier 0 · Public	`Public` (marking only)	Monitor only — no block	`CLS Retain – Public`
Tier 1 · Internal	`Internal` (footer marking)	Policy tips, no block	`CLS Retain – Internal`
Tier 2 · Confidential	`Confidential` (encrypt)	Block/restrict external — **test mode first**	`CLS Retain – Confidential`
Tier 3 · Restricted	`Restricted` (encrypt + watermark)	Block external + require justification — **test mode first**	`CLS Retain – Restricted`

*The DLP conditions (which sensitive information types trigger each tier) and the retention **periods/actions** are placeholders here — the sensitive-info-type selection is confirmed with data owners, and the retention periods are a **records/legal determination** (see §2).*

Naming convention (used in every doc and script)

Every artifact this solution creates is named `CLS <Type> – <Scope> – <Stage>` (or the tier name for labels/retention) and reused verbatim across the docs, scripts, and portal:

Artifact	Name
Sensitivity labels (the tiers)	`Public`, `Internal`, `Confidential`, `Restricted`
Sensitivity label policy (pilot)	`CLS Policy – Pilot – Ring1`
Sensitivity label policy (broad)	`CLS Policy – Broad – AllUsers`
DLP baseline policy (test mode)	`CLS DLP – Tier2-3 – Test`
Retention labels (per tier)	`CLS Retain – Public` / `– Internal` / `– Confidential` / `– Restricted`
Retention label policy (pilot)	`CLS Retain Publish – Pilot – Ring1`

What you can control

• The tier model — how many tiers, their names, definitions, and handling rules (with governance sign-off).

• The crosswalk — which label, DLP posture, and retention treatment each tier maps to.

• The classification standard document — the written scheme users and admins follow.

• The reference implementation — the tier labels, the tier-aligned DLP baseline (in test mode), and tier-aligned retention labels.

• The pilot scope and the order of enforcement (labels first, DLP and retention piloted before enforcing).

Limitations (be honest with buyers)

• **This is the framework, not the exhaustive build.** Deep auto-labeling, full DLP policy development, and the records retention schedule are separate solutions — this one establishes the scheme and a coherent reference implementation of it.

• **Licensing gates capability.** Encrypting labels are broadly available; advanced DLP, activity/content explorer reporting, and some retention features require **E5 / E5 Compliance** — **verify your licensing/tenant**; the solution flags each gate.

• **Records, retention periods, and regulatory applicability are determinations, not technical defaults** — owned by records/compliance/privacy/legal (see the §2 deferral note). This solution provides the configuration method and evidence.

• **DLP and encryption are high-impact.** A blocking DLP rule or an encrypting label applied carelessly disrupts users — always pilot in **test/simulation** first.

• **The portal is evolving.** Portal paths and some cmdlet parameters change; where a path is version-sensitive it is flagged "verify in your tenant."

The standard deployment model

> *Assess the current classification posture → design the tier model + crosswalk (the standard) → configure the reference implementation (tier labels; DLP baseline in **test**; retention labels) → pilot to a small ring → validate the tiers map coherently → expand and enforce tenant-wide. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce).***

2. Requirements Guide — Data Classification Framework & Strategy

Verify **every** requirement below before starting the Implementation Guide. These are the **solution-specific** prerequisites for building the classification framework and its reference implementation; they build on the shared **Global Requirements — Microsoft Purview Solutions** (compliance licensing, least-privilege roles, Security & Compliance PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, and a pilot scope with change control). Read the Global Requirements first; this guide does not repeat them.

> ***Deferral note (governance / records / legal).*** *The **number of tiers, their names and definitions, the handling rules, the DLP enforcement decisions, the retention periods/actions, and any regulatory applicability** referenced here are **determinations owned by your governance, compliance, records, privacy, and legal functions** — not technical defaults. This solution provides the **framework method, the crosswalk, and the configuration + evidence**; it does not decide the scheme or the schedule. Confirm every tier, condition, and period with the accountable owner before enforcing.*

2.1 Licensing for labels, DLP, and retention

• **Description:** Manual sensitivity labels, content marking, and encryption, plus baseline retention labels and DLP, are available at the Microsoft 365 E3 tier. **Advanced DLP** (some conditions, endpoint/cloud coverage), **auto-labeling**, **adaptive scopes**, and the **Data classification / activity & content explorer** reporting used for coverage require **Microsoft 365 E5** or the **E5 Compliance** add-on.

• **Why required:** The tier labels and a basic tier-aligned DLP/retention baseline can be built at E3; the coverage reporting and some advanced DLP conditions are E5-tier.

• **How to verify:** M365 admin center → subscriptions/add-ons; confirm the relevant feature is visible in the Purview portal (Information Protection, Data Loss Prevention, Data Lifecycle Management, Data classification). **Verify your licensing/tenant** — Microsoft's licensing tables are the authority.

• **How to implement if missing:** Build the framework and reference labels/retention at E3; assign E5 / E5 Compliance for the pilot before relying on advanced DLP conditions or coverage reporting.

2.2 Least-privilege roles across all three control families

• **Description:** Because the framework spans labels, DLP, and retention, the implementer needs (least privilege): **Information Protection Admin** (sensitivity labels + label policies), **Compliance Administrator** or **DLP Compliance Management** (DLP policies/rules), and **Records Management** or **Compliance Administrator** (retention labels + policies). Global Administrator is **not** required for day-to-day framework work.

• **Why required:** The cmdlets used are gated per family — `New-Label`/`New-LabelPolicy` (Information Protection), `New-DlpCompliancePolicy`/`New-DlpComplianceRule` (DLP), `New-ComplianceTag`/`New-RetentionCompliancePolicy` (retention).

• **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm the account is in the matching role groups.

• **How to implement if missing:** Add the account to the least-privilege role groups under change control; allow role propagation before running cmdlets.

2.3 Security & Compliance PowerShell connectivity

• **Description:** A workstation with the **ExchangeOnlineManagement** module (which provides `Connect-IPPSSession` for Security & Compliance PowerShell) and, where Graph is used for evidence, **Microsoft Graph PowerShell** — with a modern-auth / MFA-capable admin account.

• **Why required:** All five provided scripts and every configuration cmdlet run through `Connect-IPPSSession`.

• **How to verify:** `Connect-IPPSSession` succeeds and `Get-Label`, `Get-DlpCompliancePolicy`, and `Get-ComplianceTag` return without error.

• **How to implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in. See the Global Requirements for the full toolchain.

2.4 An agreed tier model and classification standard (governance input)

• **Description:** The **tier model** — how many tiers, their names, definitions, and handling rules — agreed with governance/compliance owners, and the **classification standard** (the written scheme) this solution will document and implement. The four-tier model in §1 is a starting point, not a mandate.

• **Why required:** Every downstream artifact (labels, DLP, retention) keys off the tier model; without an agreed scheme the crosswalk is arbitrary.

• **How to verify:** A signed-off tier list (names + definitions + handling rules) exists and the accountable owner has approved it.

• **How to implement if missing:** Facilitate the tier-model decision with governance owners before authoring the crosswalk; capture it in the standard document.

2.5 In-scope workloads and the classifying conditions per tier

• **Description:** The **workloads in scope** (Exchange, SharePoint, OneDrive, Teams; endpoints where DLP/labeling apply) and the **conditions that define each tier in data** — the sensitive information types, keywords, or classifiers that make content "Confidential" or "Restricted" for both auto-labeling later and the DLP baseline now.

• **Why required:** The DLP baseline and any auto-labeling key off the in-scope workloads and the tier conditions; without agreed conditions the tiers can't be detected in content.

• **How to verify:** Confirm the in-scope workloads and the priority sensitive info types / classifiers with data owners; validate the info types exist in Purview → *Data classification → Sensitive info types*.

• **How to implement if missing:** Define the conditions per tier before authoring the DLP baseline and before any auto-labeling engagement.

2.6 A pilot ring and a test/simulation-first commitment

• **Description:** A named **pilot ring** (a small group of users/sites) to publish the tier labels to first, and an explicit commitment that the **tier-aligned DLP baseline is created in *test* mode** (`TestWithNotifications`/`TestWithoutNotifications`) and validated before enforcement, and that **encrypting labels and any retention delete actions are piloted** before broad rollout.

• **Why required:** DLP blocking, encryption, and retention deletion are high-impact and hard to reverse cleanly; the house model requires a test/simulate pass first.

• **How to verify:** Confirm the pilot ring membership (the group backing `CLS Policy – Pilot – Ring1`), a change window, and an approver.

• **How to implement if missing:** Create the pilot group and agree the change-control path before publishing or enforcing anything.

2.7 Rights-management (encryption) prerequisites for the protected tiers

• **Description:** For the tiers whose label **encrypts** (`Confidential`, `Restricted`), the tenant's **Azure Rights Management / unified labeling protection** service must be active, and you must decide the **rights model** (which users/groups get which usage rights) per encrypting label.

• **Why required:** An encrypting label cannot protect if the protection service is not enabled; the rights definition is required input to `New-Label -EncryptionEnabled`.

• **How to verify:** Confirm the protection (Rights Management) service is **activated**; confirm the target groups for each encrypting label's rights exist.

• **How to implement if missing:** Activate the protection service and create/confirm the mail-enabled security groups used in the rights definitions before building the protected-tier labels.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are*

*in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Data Classification Framework & Strategy

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`) and **Security & Compliance PowerShell** (`Connect-IPPSSession`). Every write is run `-WhatIf` first, every high-impact policy is **piloted in a small ring**, and the **tier-aligned DLP baseline is created in *test* mode** before it is ever enforced. Artifact names follow the `CLS <Type> – <Scope> – <Stage>` convention (and the tier names) from the Overview and are reused verbatim. Portal paths are current at authoring — **verify in your tenant**, as the portal is evolving.

> ***Model:*** *Assess the posture, design the tier model + crosswalk (the standard), build the reference implementation (tier labels; DLP baseline in **test**; retention labels), pilot to a ring, validate the tiers map coherently, then **expand and enforce.** Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce). Governance/records deferral:** the tier definitions, DLP enforcement decisions, and retention periods/actions below are **placeholders** — confirm each with the accountable owner before enforcing (see §2 deferral note).*

Step 1 — Assess the current classification posture (Assess)

1. Run `CLSFRAME-Discovery.ps1` (read-only). It inventories existing **sensitivity labels** (`Get-Label`) and **label policies** (`Get-LabelPolicy`), **DLP policies** and their **mode** (`Get-DlpCompliancePolicy`), and **retention labels** (`Get-ComplianceTag`) and **retention policies** (`Get-RetentionCompliancePolicy`) — then reports which of the four framework tiers already have an artifact in each control family.

2. Review the output: what already exists, whether any DLP is already enforced (not test), and whether encryption/retention is already in use. This is your baseline — **do not** duplicate or collide with existing artifacts.

3. **Gate:** you have a written baseline of every existing label/DLP/retention object and a first view of where the framework is already partially present (so you extend, not collide).

Step 2 — Design the tier model and the crosswalk (Design)

1. With your governance owners, finalize the **tier model** (§2.4) — number of tiers, names, definitions, handling rules — using the four-tier model in the Overview as a starting point.

2. Fill in the **crosswalk**: for each tier, the sensitivity label, the DLP posture (and which sensitive info types trigger it, §2.5), and the retention treatment (period/action — records/legal sign-off). This crosswalk table **is** the classification standard's core.

3. Write the **classification standard document** (tier definitions + handling rules + crosswalk) and get it signed off.

4. **Gate:** a signed-off standard exists — tier definitions, handling rules, and a complete crosswalk (every tier has a label, a DLP posture, and a retention treatment) — before any artifact is created.

Step 3 — Build the tier sensitivity labels (Configure, -WhatIf first)

1. Run `CLSFRAME-FrameworkConfig.ps1 -WhatIf` (the default). It prints exactly which tier labels it would create with `New-Label` — `Public` (marking only), `Internal` (footer marking), `Confidential` (encrypt), `Restricted` (encrypt + watermark) — in tier order, **without making changes**.

2. Review every line: confirm each encrypting label's rights definition targets the correct group (§2.7) and the label order matches the signed-off tier model.

3. Re-run `CLSFRAME-FrameworkConfig.ps1 -Commit` to create the labels. They exist but are **not yet published** — labels only appear once a policy publishes them (Step 4).

4. **Gate:** `Get-Label` shows the four tier labels with the intended protection; none is published yet.

Step 4 — Publish the tier labels to the pilot ring (Pilot)

1. Run `CLSFRAME-FrameworkConfig.ps1 -PublishLabels -PolicyName "CLS Policy – Pilot – Ring1" -PilotGroup "<pilot-group>" -WhatIf`. It shows the `New-LabelPolicy` it would run — publishing the ordered tier label set to the pilot group only.

2. Re-run with `-Commit` to create `CLS Policy – Pilot – Ring1`.

3. In the Purview portal (**Information Protection** → **Label policies**) confirm the policy is published to the pilot group and the tier labels appear in Office for a pilot user (allow client sync time).

4. **Gate:** pilot users see the four tier labels in Word/Excel/PowerPoint/Outlook; an encrypting tier (`Confidential`/`Restricted`), applied by a pilot user, actually protects the file (open as an unauthorized user → access denied). Rollback: `Remove-LabelPolicy "CLS Policy – Pilot – Ring1"` unpublishes without deleting the labels.

Step 5 — Build the tier-aligned DLP baseline in test mode (Configure / test)

1. Run `CLSFRAME-FrameworkConfig.ps1 -BuildDlp -DlpName "CLS DLP – Tier2-3 – Test" -WhatIf`. It shows the `New-DlpCompliancePolicy` **in test mode** (`-Mode TestWithNotifications`) plus the `New-DlpComplianceRule` conditions it would create — matching the Tier 2/3 sensitive info types (and/or the `Confidential`/`Restricted` labels) and restricting external sharing. **No user action is blocked in test mode.**

2. Review the conditions against the crosswalk (§2.5) — confirm the sensitive info types and the target locations (Exchange/SharePoint/OneDrive/Teams) are correct.

3. Re-run with `-Commit` to create the DLP policy **in test mode**. Policy tips and reports are generated; nothing is blocked yet.

4. **Gate:** `Get-DlpCompliancePolicy` shows `CLS DLP – Tier2-3 – Test` with `Mode` a **Test** value (never `Enable`). Rollback: `Set-DlpCompliancePolicy "CLS DLP – Tier2-3 – Test" -Mode Disable`.

Step 6 — Build the tier-aligned retention labels (Configure, -WhatIf first)

1. Run `CLSFRAME-FrameworkConfig.ps1 -BuildRetention -WhatIf`. It shows the `New-ComplianceTag` calls for each tier retention label (`CLS Retain – Public/Internal/Confidential/Restricted`) — with the **period and action confirmed by records/legal** (§2 deferral).

2. Review each: retention action (Keep / Delete / Keep-and-Delete / disposition review), duration, and trigger (creation vs modification). **Do not** mark anything as a *record* unless records/legal has declared it so.

3. Re-run with `-Commit` to create the retention labels. They exist but are **not published** yet.

4. **Gate:** `Get-ComplianceTag` lists the four tier retention labels with the signed-off period/action; none is published.

Step 7 — Publish retention labels to the pilot ring (Pilot)

1. Run `CLSFRAME-FrameworkConfig.ps1 -PublishRetention -PolicyName "CLS Retain Publish – Pilot – Ring1" -PilotGroup "<pilot-group>" -WhatIf`, review the `New-RetentionCompliancePolicy` + `New-RetentionComplianceRule -PublishComplianceTag`, then `-Commit`.

2. Confirm pilot users can apply the tier retention labels in Office/SharePoint/Outlook.

3. **Gate:** pilot users see and can apply the tier retention labels. For any tier with a **delete** action, confirm the disposition behavior in a controlled test item before broad reliance (see Testing §5). Rollback: `Remove-RetentionCompliancePolicy "CLS Retain Publish – Pilot – Ring1"` unpublishes without deleting the labels.

Step 8 — Validate the crosswalk is coherent (Validate)

1. Run `CLSFRAME-Validation.ps1`. It checks, **per tier**, that the sensitivity label, the DLP posture, and the retention label all exist and are consistently named — i.e. every tier row in the crosswalk is fully implemented, with no orphan (a label with no retention, a DLP posture with no label, etc.).

2. Run `CLSFRAME-Crosswalk.ps1 -CsvPath .\crosswalk.csv` to export the **live** tier→label→DLP→retention crosswalk from the tenant and compare it to the signed-off standard.

3. **Gate:** the live crosswalk matches the standard — every tier has its label (published to pilot), its DLP posture (in **test**), and its retention label (published to pilot); no orphans; the DLP baseline is still in test mode. Do **not** enforce until this gate passes.

Step 9 — Measure classification coverage (Validate)

1. Run `CLSFRAME-Coverage.ps1` to report health of the framework artifacts and, where licensed, pull **labeling activity** (`Export-ActivityExplorerData`) to see how content is being classified per tier. **Activity/content explorer is E5-tier — verify your licensing/tenant.**

2. Review whether pilot content is landing in sensible tiers and whether the DLP test policy is generating the expected match reports.

3. **Gate:** the pilot shows content classifying into the tiers as designed and the DLP test policy is matching Tier 2/3 content as expected — evidence the scheme works before enforcing.

Step 10 — Expand and enforce (Expand)

1. Complete the **Testing Guide (§5)**. Only after every gate passes: re-publish the labels from `CLS Policy – Pilot – Ring1` to `CLS Policy – Broad – AllUsers` (`CLSFRAME-FrameworkConfig.ps1 -PublishLabels -PolicyName "CLS Policy – Broad – AllUsers" -AllUsers -WhatIf` → `-Commit`).

2. **Enforce the DLP baseline** only once its test-mode match reports are accepted by the data owner: `CLSFRAME-FrameworkConfig.ps1 -BuildDlp -DlpName "CLS DLP – Tier2-3 – Test" -EnforceFromTest -WhatIf`, review, then `-Commit` (flips `-Mode` from a Test value to `Enable`).

3. Widen the retention publish scope as agreed with records/legal (a deep retention schedule is a separate engagement).

4. **Gate:** all §5 tests pass; broad label publish confirmed; DLP enforced only after accepted test reports; a rollback path is documented for each policy.

Order of operations

`Assess posture → Design tier model + crosswalk (standard) → Build tier labels (-WhatIf → commit) → Publish to pilot ring → Build DLP baseline in TEST → Build retention labels → Publish retention (pilot) → Validate crosswalk coherence → Measure coverage → Expand labels broadly + enforce DLP from test`

5. Testing & Validation Guide — Data Classification Framework & Strategy

Run these tests after the Implementation Guide, **in order**. Each has an explicit **pass/fail** signal and a rollback. Do not expand from the pilot ring, and do not enforce the DLP baseline, until the relevant tests pass. All checks use the Purview portal and Security & Compliance PowerShell — the provided scripts automate the read-only ones.

> ***Reminder:*** *the tier definitions, DLP enforcement decisions, and retention periods/actions under test are the governance/records/legal determinations from §2 — you are validating that the configuration *behaves as specified*, not deciding the scheme or the schedule.*

Test 1 — Tier labels created as designed

- **How:** `CLSFRAME-Discovery.ps1`, or `Get-Label | Format-Table DisplayName,Priority`.
- **Pass:** the four tier labels (`Public`, `Internal`, `Confidential`, `Restricted`) exist in tier order, each with the intended protection (marking / encrypt / encrypt+watermark).
- **Fail:** missing/extra labels, wrong order, or wrong protection → fix in `CLSFRAME-FrameworkConfig.ps1` and re-run.

Test 2 — Tier labels visible to pilot users only

- **How:** as a **pilot** user, open Word/Excel/PowerPoint and Outlook; check the Sensitivity button lists the four tiers. Confirm a **non-pilot** user does **not** see them.
- **Pass:** pilot users see the full ordered tier set; non-pilot users see nothing (publish is scoped).
- **Fail:** labels missing for pilot users (allow client sync time) or visible to non-pilot users → re-check `CLS Policy – Pilot – Ring1` targeting.

Test 3 — Protected tiers actually encrypt

• **How:** as a pilot user, apply `Confidential` (or `Restricted`) to a test document; attempt to open it as a user **outside** the rights definition.

• **Pass:** the unauthorized user is **denied**; the authorized rights behave as defined; content marking (footer/watermark) appears where configured.

• **Fail:** unauthorized access succeeds, or rights are wrong → the protection service or the label's `EncryptionRightsDefinitions` is misconfigured (§2.7). **Do not publish broadly** until fixed.

Test 4 — DLP baseline stays in test mode until accepted

• **How:** `Get-DlpCompliancePolicy "CLS DLP – Tier2-3 – Test" | Select-Object Name,Mode`, or `CLSFRAME-Discovery.ps1`.

• **Pass:** the policy `Mode` is a **Test** value (`TestWithNotifications`/`TestWithoutNotifications`), **not** `Enable` — it reports and tips but blocks nothing.

• **Fail:** the policy is enforcing before sign-off → `Set-DlpCompliancePolicy "CLS DLP – Tier2-3 – Test" -Mode Disable` and investigate why it was enabled early.

Test 5 — DLP baseline matches the right content

• **How:** as a pilot user, send/save test content containing a Tier 2/3 sensitive info type (test data) to a location in scope; review the DLP match report / policy tip. Try clearly non-matching content too.

• **Pass:** the policy matches Tier 2/3 content (policy tip appears, match logged) and does **not** match clearly non-sensitive content.

• **Fail:** over- or under-matching → tune the DLP rule conditions (sensitive info types, instance counts) and re-test. **This is the gate before Step 10 enforcement.**

Test 6 — Tier retention labels publish and apply (pilot)

• **How:** as a pilot user, apply `CLS Retain – Internal` (or another tier) to a test item in SharePoint/Outlook; confirm it appears and is recorded.

• **Pass:** the tier retention label is available to pilot users and applies; the item shows the label.

• **Fail:** label not offered → re-check `CLS Retain Publish – Pilot – Ring1` scope and the publishing rule (`New-RetentionComplianceRule -PublishComplianceTag`).

Test 7 — Retention *behavior* is safe (controlled disposition test)

• **How:** for any tier retention label with a **delete** action, apply it to a **disposable test item** with a short/adjusted trigger in a controlled area (never production records) and confirm the disposition/disposition-review behaves as specified.

• **Pass:** at end of period the item is retained/disposed/sent to review exactly as the records/legal determination specifies.

• **Fail:** unexpected deletion or no disposition → **halt broad reliance**, review the label action/trigger with records/legal before enforcing anywhere real.

Test 8 — The crosswalk is coherent (no orphans)

- **How:** `CLSFRAME-Validation.ps1`.
- **Pass:** every tier has its sensitivity label **and** a DLP posture **and** a retention label, consistently named — no orphan (a label with no retention, a DLP posture with no matching tier label, etc.).
- **Fail:** any tier row incomplete or misnamed → build the missing artifact (`CLSFRAME-FrameworkConfig.ps1`) and re-validate.

Test 9 — The live crosswalk matches the standard

- **How:** `CLSFRAME-Crosswalk.ps1 -CsvPath .\crosswalk.csv`; compare the exported tier→label→DLP→retention table to the signed-off standard document.
- **Pass:** the live crosswalk equals the standard — same tiers, same mappings, DLP still in test.
- **Fail:** drift between live config and the standard → reconcile (fix config or update the standard with governance sign-off) before expanding.

Test 10 — Coverage evidence before enforcement

- **How:** `CLSFRAME-Coverage.ps1` (activity/content explorer is E5-tier — **verify your licensing/tenant**).
- **Pass:** pilot content is classifying into the tiers as designed and the DLP test policy is matching Tier 2/3 content as expected — the scheme demonstrably works.
- **Fail:** content not classifying sensibly or DLP not matching → tune conditions and re-measure before enforcing.

Test 11 — Full readiness before expansion

- **How:** `CLSFRAME-Validation.ps1` (re-run) after all fixes.
- **Pass:** tier labels published to pilot, DLP baseline in **test** with accepted match reports, retention labels published to pilot, crosswalk coherent and matching the standard — all green.
- **Fail:** any red → resolve before publishing `CLS Policy – Broad – AllUsers` or enforcing the DLP baseline.

Exit criteria (all must be true before broad enforcement)

`Four tier labels built as designed · pilot users see labels · protected tiers encrypt · DLP baseline in test only · DLP matches correct content · retention applies + disposition safe · crosswalk coherent (no orphans) · live crosswalk == standard · coverage evidence positive · rollback documented per policy`

6. Troubleshooting Guide — Data Classification Framework & Strategy

Common failure modes when standing up the classification framework, with the symptom, the likely cause, and the resolution. Work top-down; most issues are connectivity, role, licensing, or publish-scope problems. All fixes stay inside the **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand** model — never enforce to resolve a test problem.

> ***Determinations stay deferred.*** *If a fix implies changing a tier definition, a DLP enforcement decision, or a retention period/action, that is a* ***governance/records/legal*** *call (see §2) — the steps below only correct the *technical* configuration.*

Connectivity & permissions

`Connect-IPPSSession` **/** `New-Label` **/** `Get-DlpCompliancePolicy` **is not recognized**

- **Cause:** the ExchangeOnlineManagement module is not installed, or the session is not connected.
- **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`, then `Connect-IPPSSession`

with a modern-auth / MFA admin. Re-run `Get-Label` to confirm the session is live. The scripts call `Connect-IPPSSession` if no session is detected — accept the sign-in prompt.

Cmdlet returns "The operation couldn't be performed because ... access denied" / no results

- **Cause:** the account lacks the least-privilege role for that control family (§2.2).
- **Fix:** confirm membership in **Information Protection Admin** (labels), **Compliance

Administrator / DLP Compliance Management **(DLP), and** Records Management / Compliance Administrator **(retention) in Purview** → Roles & scopes → Permissions**. Allow role propagation (can take time) before retrying.

A cmdlet parameter is rejected (e.g. `-Mode`**,** `-EncryptionEnabled`**)**

- **Cause:** the S&C PowerShell surface evolved, or the module is out of date.
- **Fix:** `Update-Module ExchangeOnlineManagement`; verify the parameter with `Get-Help New-Label

-Full`. Portal paths and some parameters change — **verify in your tenant**; do not force a parameter the current module does not expose.

Labels not visible or not protecting

Tier labels don't appear in Office for pilot users

- **Cause:** labels are created but **not published**, the user isn't in the pilot group, or the

client hasn't synced.

- **Fix:** confirm `CLS Policy – Pilot – Ring1` is published to the pilot group (Purview →

Information Protection → **Label policies**); confirm the user is a member; allow client sync time (sign out/in of Office). Labels only surface once a policy publishes them — creating a label is not enough.

Labels visible to users who are NOT in the pilot

- **Cause:** the label policy is scoped to All users instead of the pilot group.

- **Fix:** re-scope `CLS Policy – Pilot – Ring1` to the pilot group only; do not use

`-PublishLabels -AllUsers` until Step 10. Rollback: `Remove-LabelPolicy "CLS Policy – Pilot – Ring1"` unpublishes without deleting the labels.

An encrypting tier (`Confidential`/`Restricted`) does not deny an unauthorized user

- **Cause:** the protection (Azure Rights Management) service is not activated, or the label's

`EncryptionRightsDefinitions` grant too broadly (§2.7).

- **Fix:** confirm the protection service is **active**; review the rights definition targets the

intended group only; re-apply the label to a fresh test file (already-protected files keep their original policy). **Do not publish broadly** (Step 10) until Test 3 passes.

DLP baseline issues

`CLS DLP – Tier2-3 – Test` **is blocking / enforcing before sign-off**

- **Cause:** the policy `Mode` was set to `Enable` prematurely.
- **Fix:** `Set-DlpCompliancePolicy "CLS DLP – Tier2-3 – Test" -Mode Disable`, investigate why it was

enabled, then return it to a **Test** value. Enforcement is Step 10 only, and only after the data owner accepts the test-mode match reports (Test 5).

DLP over-matches or under-matches

- **Cause:** the rule's sensitive-info-type conditions or instance/confidence thresholds don't align

to the tier conditions (§2.5).

- **Fix:** tune `New-DlpComplianceRule` conditions (info types, min/max instance counts, confidence),

re-run in **test**, and re-check the match report. Keep iterating in test — never tune by enabling.

No DLP match reports appear

- **Cause:** test content didn't reach an in-scope location, or reporting latency.
- **Fix:** confirm the policy locations (Exchange/SharePoint/OneDrive/Teams) include where you tested;

allow reporting latency; use clearly-matching **test** data (never real regulated data) for the sensitive info type.

Retention issues

Tier retention labels not offered to pilot users

- **Cause:** the retention labels are created (`New-ComplianceTag`) but the publish policy

(`CLS Retain Publish – Pilot – Ring1`) isn't scoped to the pilot or hasn't propagated.

- **Fix:** confirm `New-RetentionComplianceRule -PublishComplianceTag` published the tier tags to the

pilot scope; allow propagation (retention publishing can take time to surface in SharePoint/Outlook).

A retention label with a delete action behaved unexpectedly in testing

- **Cause:** the action/trigger (Keep/Delete, creation vs modification, period) doesn't match the intended records/legal determination.
- **Fix: halt broad reliance immediately**; review the label's action and trigger with records/legal (§2 deferral) before applying it anywhere real. Test 7 must pass on a disposable item before any tier with a delete action is trusted.

Crosswalk / coverage

`CLSFRAME-Validation.ps1` **reports an orphan tier**

- **Cause:** a tier has a label but no retention (or a DLP posture with no matching tier label, etc.) — the crosswalk row is incomplete.
- **Fix:** build the missing artifact with `CLSFRAME-FrameworkConfig.ps1` for that tier, then re-validate. Do not expand until every tier row is complete and consistently named.

Live crosswalk (`CLSFRAME-Crosswalk.ps1`) drifts from the signed-off standard

- **Cause:** config changed outside the standard, or the standard changed without updating config.
- **Fix:** reconcile — either correct the tenant config or update the standard **with governance sign-off**. The live crosswalk must equal the standard before expansion (Test 9).

`CLSFRAME-Coverage.ps1` **returns no activity data**

- **Cause:** activity/content explorer (`Export-ActivityExplorerData`) is **E5-tier** and may be unlicensed, or there's simply no pilot activity yet.
- **Fix: verify your licensing/tenant**; the script degrades to a health-only report when the activity API is unavailable. Generate pilot labeling activity, then re-run.

If a step's gate can't be met

Stop at that gate — do **not** work around it by expanding scope or enabling enforcement. Re-run the relevant read-only script (`Discovery`, `Validation`, `Crosswalk`, `Coverage`), capture the evidence, and resolve the specific technical cause above (or escalate a *determination* to the accountable governance/records/legal owner) before proceeding.

7. Best Practices — Data Classification Framework & Strategy

Field-tested practices for building a classification framework that the whole organization can align to **without a user revolt**. These complement the Implementation and Testing guides.

Tier model design

- **Keep the tiers few.** Three to four tiers is the practical ceiling — `Public / Internal / Confidential / Restricted` covers most organizations. More tiers and users guess or ignore; the

tier *count* is a usability decision, not a technical one.

• **Define each tier in plain language, with a handling rule.** "Confidential — sensitive business data or PII; encrypt and don't share externally" beats an abstract label. The tier definition is the user's only guidance at the moment of classifying.

• **One scheme, one source of truth.** The whole point of a framework is that labels, DLP, and retention all mean the same thing by tier. Resist per-team or per-workload tier variants — they reintroduce the inconsistency the framework exists to remove.

• **Design the tier model once, with governance owners.** Renaming or re-tiering after publish is disruptive (existing content keeps the old label). Get the tier model and standard signed off before creating any artifact.

The crosswalk (the core discipline)

• **Every tier maps to exactly one label, one DLP posture, and one retention treatment.** The crosswalk table is the contract; a complete row per tier is what makes coverage provable.

• **Hunt for orphans.** A label with no retention, a DLP rule with no matching tier, a retention label no tier points to — each is a gap or a source of confusion. `CLSFRAME-Validation.ps1` exists to catch them; run it whenever the config changes.

• **Keep the live crosswalk equal to the standard.** When someone adds a policy outside the framework, the standard and the tenant drift. Re-export the live crosswalk (`CLSFRAME-Crosswalk.ps1`) on a cadence and reconcile.

Protection & DLP

• **Encrypt only the tiers that need it.** `Public`/`Internal` should mark (footer) but not encrypt — visible classification with zero friction. Reserve encryption for `Confidential`/`Restricted`, and always pilot first.

• **Define encryption rights by group, not by person.** Point `EncryptionRightsDefinitions` at mail-enabled security groups so membership changes don't require editing the label.

• **Always build the DLP baseline in *test* mode first.** `TestWithNotifications`/`TestWithoutNotifications` reports and tips without blocking. Enforcing a noisy DLP policy is the fastest way to lose user trust — measure precision, then enforce.

• **Align DLP conditions to the tier definitions.** The Tier 2/3 DLP rule should trigger on the *same* sensitive info types that make content Tier 2/3 in the standard — otherwise DLP and the labels disagree.

Retention (defer the determination)

• **Retention periods and records declarations are records/legal's call.** Configure exactly what the accountable owner signs off; never invent a schedule or declare a record without their determination. This framework provides the tier→retention mapping method, not the periods.

• **Pilot delete actions on disposable content only.** Any tier retention label that deletes must be proven on throwaway test items with a controlled trigger before it goes near real records.

• **The deep retention schedule is a separate engagement.** Here you establish the tier-aligned retention *labels*; the full records file plan and schedule are their own solution.

Program & change management

• **Pilot ring, always.** Publish tier labels to `CLS Policy – Pilot – Ring1` before `CLS Policy – Broad – AllUsers`; the pilot surfaces marking, encryption, and DLP-tip issues cheaply.

• **Document a rollback per policy.** Unpublish (`Remove-LabelPolicy` / `Remove-RetentionCompliancePolicy`) or `Set-DlpCompliancePolicy -Mode Disable` removes exposure without deleting artifacts; know the exact command before you publish.

• **Communicate the scheme before rollout.** Tell users what the tiers mean, how to pick one, and where to get help — the ticket flood comes from surprise, not from tiers. The classification standard document is the communication asset.

• **Keep least-privilege roles.** Information Protection Admin, DLP Compliance Management, and Records Management cover the framework; don't grant Global Admin for day-to-day work.

• **Re-baseline periodically.** Run `CLSFRAME-Discovery.ps1` and `CLSFRAME-Validation.ps1` on a cadence to catch label sprawl, orphaned policies, and framework drift.

Security & safety hygiene

• **No credentials in code.** All scripts authenticate interactively via `Connect-IPPSSession` / `Connect-MgGraph`; never embed secrets. Store any exports securely.

• **Read-only or `-WhatIf` by default.** Every provided script defaults to no-change; writes require an explicit `-Commit`, and the DLP baseline is created in **test** mode. Keep it that way.

• **Verify licensing/tenant before relying on advanced features.** Advanced DLP conditions, activity/content explorer coverage reporting, and auto-labeling are plan-gated and the portal evolves — flag, don't assume.

8. Deliverables Summary — Data Classification Framework & Strategy

What you receive

A complete, self-guided package to design and stand up an **organization-wide data classification framework** in Microsoft Purview — a small set of sensitivity tiers, a written classification standard, and a **crosswalk** mapping each tier to its sensitivity label, DLP posture, and retention treatment — configured as a coherent reference implementation, piloted in test/simulation, and validated, using only the Microsoft Purview portal and Security & Compliance PowerShell.

Documentation

Document	What it gives you
Solution Overview	The tier model, the classification standard concept, the tier→label→DLP→retention crosswalk, the naming convention, scope boundaries vs. the deep build solutions, and the deployment model.
Requirements Guide	Solution-specific prerequisites (licensing across labels/DLP/retention, least-privilege roles for all three families, S&C PowerShell, an agreed tier model, per-tier conditions, encryption prerequisites, pilot ring) on top of the Purview Global Requirements.
Implementation Guide	Ordered steps: assess posture → design tier model + crosswalk (the standard) → build tier labels (`-WhatIf`→commit) → publish to pilot → build DLP baseline in **test** → build + publish retention labels → validate crosswalk coherence → measure coverage → expand + enforce. Each step named, gated, and reversible.
Testing & Validation Guide	11 pass/fail tests — tier labels, visibility, encryption, DLP-in-test, DLP matching, retention application + safe disposition, crosswalk coherence (no orphans), live-vs-standard, and coverage evidence.
Best Practices	Tier-model design, the crosswalk discipline, protection/DLP, retention deferral, program/change management, and safety guidance.
Troubleshooting Guide	The common failure modes (cmdlet not recognized, labels not visible, encryption not enforcing, DLP enabled too early, orphan tiers, retention not applying) with resolutions.

PowerShell tools (5)

All run through `Connect-IPPSSession` (Security & Compliance PowerShell); read-only or `-WhatIf` by default, writes gated behind `-Commit`, and the DLP baseline created in **test** mode. No credentials in code.

Script	Role	Default
`CLSFRAME-Discovery.ps1`	**Posture assessment** — inventory existing sensitivity labels, label policies, DLP policies + mode, and retention labels/policies, and report which framework tiers already have an artifact per control family.	Read-only

Script	Role	Default
`CLSFRAME-FrameworkConfig.ps1`	**Framework config/remediation helper** — create the tier labels, publish the label policy (pilot → broad), build the tier-aligned DLP baseline **in test mode** (then enforce from test), and create + publish tier retention labels.	`-WhatIf`
`CLSFRAME-Crosswalk.ps1`	**Crosswalk reporting** — export the live tier→label→DLP→retention crosswalk from the tenant (CSV/console) to compare against the signed-off standard.	Read-only
`CLSFRAME-Validation.ps1`	**Coherence validation** — check, per tier, that the label, DLP posture, and retention label all exist and are consistently named (no orphans); DLP still in test.	Read-only
`CLSFRAME-Coverage.ps1`	**Scope/health + coverage helper** — health-check the framework artifacts and, where licensed, pull labeling activity (`Export-ActivityExplorerData`) to measure how content classifies per tier.	Read-only

Outcomes

• A signed-off **classification standard** — tier definitions, handling rules, and a complete crosswalk — that DLP, labels, and retention all align to.

• The **tier labels** built and published to a pilot ring, with encryption proven to enforce on protected tiers.

• A **tier-aligned DLP baseline** validated in **test** mode before any enforcement.

• **Tier-aligned retention labels** published to the pilot, with disposition proven safe.

• Repeatable read-only tooling to export the live crosswalk, validate coherence, and re-baseline the framework over time.

Explicitly out of scope (deferred)

• **The tier model itself, DLP enforcement decisions, retention schedules, records declarations, and regulatory applicability** — determinations owned by governance/compliance/records/privacy/legal; this solution provides the framework method, the crosswalk, and the configuration + evidence, not the determination.

• **Deep build solutions** — the exhaustive auto-labeling build, full DLP policy development, and the records retention file plan/schedule are separate Purview solutions that plug into this framework.

• Any capability your **licensing/tenant** does not include — verify before relying on advanced DLP conditions or activity/content explorer coverage reporting.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLSFRAME-Coverage.ps1

CLSFRAME-Crosswalk.ps1

CLSFRAME-Discovery.ps1

CLSFRAME-FrameworkConfig.ps1

CLSFRAME-Validation.ps1